

AIPSA-GM: Adaptive Islanded Parallel Simulated Annealing with Guided Migration

Pengxiang Li, Weilun Qiu, Jinkuo Ha

I. INTRODUCTION & MOTIVATION

Simulated Annealing (SA) is a widely used stochastic optimization algorithm inspired by thermodynamic annealing processes. It has been successfully applied to a variety of real-world problems, including logistics optimization, circuit design, and scheduling. Despite its effectiveness, SA is inherently sequential due to its Markov chain structure, where each state depends on the previous one. This sequential dependency makes it challenging to fully exploit modern parallel hardware.

The limitations of serial SA become particularly evident in large-scale problems. The algorithm often converges slowly and is prone to getting trapped in local optima. Parallelization through island models provides a promising direction, where multiple independent SA instances evolve concurrently and periodically exchange solutions. However, this approach introduces several challenges, including balancing exploration and exploitation, managing communication overhead, and mitigating synchronization costs.

To address these issues, we propose AIPSA-GM (Adaptive Islanded Parallel Simulated Annealing with Guided Migration). Our approach integrates several key innovations, including a quality-first multiplicative migration strategy, phase-aware adaptive temperature control, pool-normalized diversity, asynchronous buffered migration, and a GPU-based parallel extension. Together, these components aim to improve both optimization performance and parallel efficiency.

II. RELATED WORK

The concept of Simulated Annealing was first introduced in [1] Kirkpatrick et al. (1983), building upon the Metropolis acceptance criterion proposed in [2]. Since then, numerous efforts have been made to parallelize SA, particularly through island models that distribute computation across multiple processes. Prior work [3] has shown that communication topology plays a critical role in determining convergence behavior and scalability. In addition, GPU-based implementations [4] have been explored to accelerate SA by leveraging massive data parallelism. Our work builds upon these foundations by combining algorithmic and system-level improvements into a unified framework.

III. TECHNICAL APPROACH

To support experimentation across different optimization problems, we define a unified problem interface consisting of four core functions: initialization of a solution, generation of a neighboring solution, evaluation of cost, and computation of distance between solutions. This abstraction decouples

the optimization logic from problem-specific implementations. For example, in the Traveling Salesman Problem (TSP), we use a 2-opt neighborhood structure and edge-based distance, whereas in the Rastrigin function, we use Gaussian perturbations and Euclidean distance.

As a baseline, we implement a standard serial SA algorithm that includes a geometric cooling schedule, an acceptance rule based on the Metropolis criterion $\exp(-\Delta f/T)$, and a stopping condition based on either temperature or iteration limits. This baseline provides a reference point for evaluating the effectiveness of parallelization.

We further implement two parallel baselines. The first, referred to as independent replicas, runs multiple SA instances without any communication. While this approach achieves maximum parallelism, it often suffers from poor solution quality due to lack of information sharing. The second baseline adopts a synchronous island model, where islands periodically synchronize through a global barrier and exchange solutions. Although this improves convergence, it introduces idle time and limits scalability.

Our proposed method, AIPSA-GM, extends these baselines with several key innovations. First, we introduce a guided migration strategy based on a multiplicative utility function:

$$U = q \times (1 + \beta d),$$

where q represents normalized solution quality and d represents diversity relative to the receiving island. Unlike the additive formulation $U = \alpha q + \beta d$, this multiplicative form ensures that low-quality solutions cannot be selected regardless of their diversity score, making quality a hard prerequisite while diversity serves as a secondary tie-breaker. Diversity is measured relative to the current migration pool rather than the global search space, providing stable $[0, 1]$ scores regardless of problem scale or dimensionality.

Second, we implement adaptive temperature control based on the acceptance rate. The temperature is increased when the acceptance rate is too low, decreased when it is too high, and otherwise follows the standard cooling schedule:

$$T = \begin{cases} 1.1T & r < r_{low} \\ 0.9T & r > r_{high} \\ \alpha T & \text{otherwise.} \end{cases}$$

To further improve stability, we introduce auto-calibrated cooling, where the cooling rate α is computed such that the

temperature reaches the minimum value exactly at the final iteration:

$$\alpha = \left(\frac{T_{min}}{T_0} \right)^{1/N}.$$

To prevent runaway reheating in later stages, we adopt a phase-aware design: reheating is restricted to the first 60% of iterations, the reheat factor decays linearly from 1.10 to 1.02 with iteration progress, and temperature is capped at $0.3 \times T_0$.

Finally, we design an asynchronous migration mechanism in which islands communicate through non-blocking queues. This eliminates the need for global synchronization and allows computation and communication to overlap, significantly improving efficiency.

Algorithm 1 AIPSA-GM Island Worker (simplified)

```

1: Initialize  $x, x^*, T \leftarrow T_0$ ; auto-calibrate  $\alpha$ 
2: while  $T > T_{min}$  and  $iter < \max\_iter$  do
3:   for  $l = 1$  to  $L$  do
4:     Apply Metropolis criterion; update  $x^*$  if improved
5:     Adjust  $T$  adaptively (phase-aware)
6:     if  $iter \% migration\_interval = 0$  then
7:       Push  $x^*$  to neighbors
8:       Select and accept migrant by  $U = q(1 + \beta d)$ 
9:     end if
10:  end for
11:  Pull migrants at cooling boundary;  $T \leftarrow T \times \alpha$ 
12: end while
13: return  $x^*$ 
```

From a complexity perspective, each SA iteration requires $O(n)$ work for TSP (2-opt swap) and $O(d)$ for Rastrigin (d dimensions). With k islands, AIPSA-GM achieves $O(N \cdot n/k)$ wall-clock complexity for the computation phase, with an additional $O(k)$ migration cost per interval under ring topology and $O(k^2)$ under fully connected topology. For the GPU extension, the `dist_matrix` requires $O(n^2)$ space for TSP, which saturates memory bandwidth at large chain counts and determines the optimal parallelism level.

In addition to CPU-based parallelism, we develop a GPU-based extension. For the Rastrigin benchmark, we implement a multi-chain SA kernel in which each CUDA thread executes an independent SA trajectory. For TSP, we implement a Boltzmann-weighted 2-opt strategy that preserves the probabilistic acceptance behavior of SA while enabling parallel execution across multiple chains.

IV. PARALLELIZATION DESIGN

The parallelization strategy of AIPSA-GM is built upon the island model, where multiple simulated annealing (SA) processes evolve concurrently and interact through periodic migration. While this model provides a natural form of task-level parallelism, achieving both high efficiency and strong solution quality requires careful consideration of data dependencies, load balancing, synchronization, and communication overhead.

A. Data Dependencies

A key advantage of the island model is that each island evolves largely independently, with interactions occurring only during migration events. Within each island, the SA process follows a Markov chain in which each state depends only on the previous state. This sequential dependency prevents fine-grained parallelization within a single chain but enables coarse-grained parallelism across multiple islands.

As a result, data dependencies are minimal and localized. Each island maintains its own solution, temperature schedule, and acceptance statistics, allowing independent execution without shared-state contention. Migration introduces occasional dependencies when solutions are exchanged, but these interactions are infrequent and do not require continuous synchronization. This design allows AIPSA-GM to exploit parallel hardware efficiently while maintaining the correctness of the SA process.

B. Load Balancing

Load balancing is a non-trivial challenge in stochastic optimization algorithms such as SA. Due to the randomness in neighbor generation and acceptance decisions, different islands may experience significantly different convergence trajectories and computational workloads. For example, an island trapped in a region with frequent rejections may progress faster than one exploring a highly variable region with many accepted moves.

In synchronous parallel designs, such variability leads to idle time, as faster processes must wait at synchronization barriers for slower ones. This effect becomes more pronounced as the number of islands increases, reducing overall parallel efficiency.

To mitigate this issue, AIPSA-GM adopts an asynchronous execution model. Each island proceeds at its own pace without waiting for others, allowing computational resources to be utilized continuously. Migration is handled opportunistically through non-blocking communication, ensuring that load imbalance does not translate into wasted compute cycles. This design significantly improves scalability, particularly in heterogeneous or large-scale settings.

C. Synchronization

Synchronization is a major source of inefficiency in parallel systems. In traditional synchronous island models, all processes must reach a global barrier before migration can occur. This barrier introduces idle time and limits scalability, especially when the workload is uneven or when communication latency is non-negligible.

AIPSA-GM eliminates global synchronization by employing asynchronous buffered migration. Instead of waiting for all islands to reach a synchronization point, each island independently sends and receives solutions through non-blocking queues. Migration checks are performed periodically during execution, but they do not interrupt the main computation flow.

This design allows communication and computation to overlap, reducing idle time and improving resource utilization. However, it also introduces weaker consistency across islands, as different islands may operate on slightly outdated information. In practice, we find that this tradeoff is beneficial, as the increased throughput outweighs the loss of strict synchronization.

D. Communication Costs

Communication overhead is a critical factor in determining the scalability of parallel algorithms. In AIPSA-GM, communication occurs during migration events, where solutions are exchanged between islands. The cost of communication depends on both the migration frequency and the topology of connections between islands.

We analyze three common topologies: ring, fully connected, and random k -neighbor. In a ring topology, each island communicates only with its immediate neighbors, resulting in $O(n)$ communication complexity. This structure preserves diversity by limiting information flow but may slow convergence due to reduced dissemination of high-quality solutions.

In contrast, a fully connected topology allows every island to communicate with all others, leading to $O(n^2)$ communication complexity. While this promotes rapid convergence by quickly spreading good solutions, it can cause significant communication overhead and reduce scalability at large island counts.

The random k -neighbor topology provides a compromise between these extremes by limiting each island to a fixed number of randomly selected neighbors. This reduces communication cost while maintaining some level of global information exchange.

In practice, the optimal topology is problem-dependent. For combinatorial problems such as TSP, rapid convergence is desirable, making fully connected topologies more effective. For multimodal continuous problems such as Rastrigin, preserving diversity is more important, making sparse topologies like ring preferable. Understanding this tradeoff is essential for designing scalable parallel SA systems.

V. IMPLEMENTATION DETAILS

Our implementation is built on a hybrid CPU-GPU architecture. For CPU-based parallelism, we use Python multiprocessing to implement the island model, where each process corresponds to an independent simulated annealing instance. For GPU acceleration, we utilize CUDA through the CuPy RawModule interface, enabling custom kernel execution for massively parallel SA chains.

All experiments are conducted on a system equipped with an Intel i9 CPU and an NVIDIA RTX 4090 GPU under a WSL2 environment with CUDA 12.6. The implementation is fully modular, allowing consistent experimentation across different problem settings and algorithm variants. To ensure reproducibility, all experiments are executed with fixed random seeds and automated scripts `run_all.sh`. The complete source code and experiment scripts are publicly available at <https://github.com/pengxiangli666/AIPSA-GM>.

A. Key Parameters

The main parameters used across all experiments are summarized in Table I. These parameters are chosen to balance convergence quality and computational efficiency, and are kept consistent across different experimental settings unless otherwise specified.

TABLE I
KEY PARAMETERS FOR AIPSA-GM

Parameter	Value	Description
T_0	5000.0	Initial temperature
α	0.995	Base cooling rate*
$T_{\min}$	10^{-3}	Minimum temperature threshold
L	100	Inner loop length per temperature
Migration Interval	300	Steps between migration events
Number of Islands	4	Parallel SA instances

* Fixed at 0.995 for Serial SA and Baselines; auto-calibrated for AIPSA-GM.

VI. EXPERIMENTAL SETUP

We evaluate our approach on two benchmark problems: the Traveling Salesman Problem and the Rastrigin function. The TSP experiments range from 1000 to 10,000 cities, while the Rastrigin experiments range from 10 to 1000 dimensions.

Each configuration is evaluated across 10 independent runs with different random seeds, except for GPU Rastrigin experiments which use 5 runs due to longer per-run runtime. Results are reported using metrics such as mean cost, best cost, standard deviation, and runtime. This ensures that our evaluation captures both solution quality and robustness.

VII. RESULTS

In this section, we present the empirical results of AIPSA-GM across a series of experiments designed to evaluate different components of the algorithm. The experiments are conducted on both the Traveling Salesman Problem (TSP) and the Rastrigin function to cover combinatorial and continuous optimization settings.

Given the stochastic nature of simulated annealing, each configuration is evaluated over multiple independent runs with different random seeds. We report standard metrics including mean cost, best cost, standard deviation, and wall-clock runtime to summarize performance.

Each subsection corresponds to a specific experiment and presents the observed trends through figures and quantitative comparisons in the Appendix section. We focus on describing the empirical behavior and key performance differences between methods, while a deeper interpretation of these results is provided in the following Analysis and Discussion section.

A. Experiment 1: Is Parallel SA Beneficial?

Figure 1 and 2 show the comparison of different simulated annealing variants on both benchmarks. On TSP, Serial SA and Baseline A achieve similar performance, while Baseline B significantly reduces the cost. AIPSA-GM further improves the result, achieving the lowest mean cost with a 37.4% reduction

over Serial SA and 13.0% over Baseline B, along with slightly lower variance.

A similar trend is observed on Rastrigin. Serial SA performs the worst with high variance, while Baseline A and B progressively improve the solution quality. AIPSA-GM again achieves the best performance, reducing the mean cost by 79.7% compared to Serial SA and showing improved stability across runs.

B. Experiment 2: Migration Policy

Figure 3, 4 and 5 compare different migration policies across TSP and Rastrigin benchmarks. On TSP, all migration-based methods outperform random selection, with *quality_only* achieving the lowest mean cost. The performance gap between *guided* and *quality_only* is small, indicating comparable effectiveness on this problem.

On Rastrigin with 10 dimensions, the differences between policies are less pronounced. While *best_only* achieves the lowest mean cost, all methods exhibit similar performance with overlapping error bars, suggesting limited sensitivity to migration strategy in this setting.

In contrast, on Rastrigin with 30 dimensions, *guided* migration achieves the best performance, improving over both *random* and *best_only*. The gap between policies becomes more visible compared to the 10D case, with *guided* consistently achieving lower mean cost.

Figure 6 further compares the effect of topology on Rastrigin 30D. Under full topology, the performance differences between policies are less distinct, whereas under ring topology, *guided* migration achieves a clearer advantage over other methods.

C. Experiment 3: Adaptive Temperature

Figure 7 shows the comparison between fixed and adaptive cooling on the Rastrigin benchmark. Adaptive cooling significantly improves performance, reducing the mean cost by 47.2% compared to fixed cooling, while also achieving lower variance across runs.

In contrast, Figure 8 presents the results on the TSP benchmark, where fixed cooling consistently outperforms adaptive cooling. The adaptive variant results in higher mean cost, with fixed cooling achieving approximately 25% better performance. The variance is also slightly lower for the fixed schedule.

These results show that the effectiveness of adaptive temperature differs across problem settings, with opposite trends observed on the two benchmarks.

D. Experiment 4: Async vs Sync

Figure 9 shows the comparison between synchronous and asynchronous migration on the TSP benchmark. The asynchronous variant achieves both lower mean cost and slightly faster runtime, improving solution quality by 12.9% while providing a modest speedup.

On the Rastrigin benchmark (Figure 10), asynchronous migration again achieves better solution quality, reducing the

mean cost by 44.6% compared to the synchronous variant. However, this improvement comes with increased runtime, as the asynchronous version runs longer in this setting.

Figure 11 further shows how the advantage of asynchronous migration changes with the number of islands on TSP. The improvement over the synchronous baseline increases as the number of islands grows from 2 to 8, and then stabilizes at higher island counts.

Overall, asynchronous migration consistently achieves better solution quality across both benchmarks, with varying impact on runtime depending on the problem setting.

E. Experiment 5: Topology & Scalability

Figure 12 and Figure 13 compare different communication topologies on TSP and Rastrigin benchmarks. On TSP, the fully connected topology achieves the lowest mean cost, outperforming both ring and random- k topologies. In contrast, on Rastrigin, the ring topology achieves the best performance, while the fully connected and random- k variants result in higher mean costs. These results indicate that the preferred topology differs across problem settings.

Figure 14 shows the scalability behavior of different topologies on the Rastrigin benchmark as the number of islands increases. While all methods improve in solution quality with more islands, the runtime of the fully connected topology increases significantly at higher island counts, whereas ring and random- k topologies exhibit more stable runtime growth.

Figure 15 presents scalability results on TSP across all solvers. As the number of islands increases, AIPSA-GM consistently achieves lower mean cost, with the improvement over the serial baseline increasing from approximately 28% to 44%. Baseline methods show smaller gains, while the serial baseline remains unchanged.

F. Experiment 6: GPU Parallelism

Figure 16 shows the quality gain of GPU over CPU on the Rastrigin benchmark across different dimensionalities. The GPU consistently outperforms the CPU, with a substantial improvement at low dimensions (67.4% at 10D). As the dimensionality increases, the improvement gradually decreases, reaching 9.1% at 1000 dimensions.

On the TSP benchmark (Figure 17), GPU also achieves consistent improvements over CPU across all problem sizes. The gain is relatively modest for smaller instances (around 5–9%), but increases for larger instances, reaching up to 17.6% at 10,000 cities.

Figure 18 presents representative cost comparisons at selected scales, confirming that GPU consistently achieves lower mean cost than CPU on both benchmarks. Figure 19 further shows the effect of the number of parallel chains, where performance varies with different chain counts. Finally, Figure 20 illustrates the throughput advantage of GPU over CPU, showing that GPU executes significantly more iterations within the same time budget across all tested dimensions.

Overall, GPU-based parallelism consistently improves solution quality across both benchmarks, with the magnitude of improvement varying across problem scales.

VIII. ANALYSIS & DISCUSSION

A. Migration as the Key Driver of Performance

The results from Experiment 1 and Experiment 2 highlight that parallelism alone is insufficient to achieve significant performance gains. In particular, the independent replicas baseline performs similarly to Serial SA, indicating that simply running multiple chains in parallel does not effectively improve solution quality. Instead, the introduction of migration plays a critical role in enabling performance improvements.

Furthermore, Experiment 2 shows that the choice of migration policy directly impacts solution quality. While all migration-based approaches outperform random selection, the relative performance varies across problem settings. In high-dimensional Rastrigin, guided migration achieves the best performance, whereas in TSP, its advantage over quality-only selection is marginal. This suggests that the benefit of incorporating diversity into migration depends on the structure of the search space.

B. Problem-Dependent Behavior

A consistent pattern across multiple experiments is that algorithmic design choices exhibit strong problem dependence. This is most clearly demonstrated in Experiment 3, where adaptive temperature significantly improves performance on Rastrigin but degrades performance on TSP. Similarly, in Experiment 5, the optimal communication topology differs between the two benchmarks: fully connected topology performs best for TSP, while ring topology is more effective for Rastrigin.

These observations indicate that continuous multimodal problems and combinatorial optimization problems respond differently to exploration and information sharing mechanisms. As a result, a single fixed design choice may not be optimal across different problem domains.

C. Impact of Asynchronous Execution

Experiment 4 demonstrates that asynchronous migration consistently improves solution quality compared to synchronous execution. This improvement stems from the elimination of global synchronization barriers, which leads to better resource utilization and improved scalability.

The scalability results further show that the advantage of asynchronous migration increases with the number of islands. As the system scales, the cost of synchronization becomes more pronounced, and eliminating barriers leads to greater performance gains. This highlights the importance of system-level design in parallel optimization algorithms.

D. Topology and Scalability Tradeoffs

Experiment 5 reveals a clear tradeoff between solution quality and communication cost across different topologies. Fully connected topology enables rapid information sharing and achieves strong performance on TSP, but exhibits significant runtime overhead at larger island counts. In contrast, ring topology provides more stable scalability, with slower

growth in communication cost and more consistent performance across different scales.

These results demonstrate that communication structure has a direct impact on both convergence behavior and scalability. Choosing an appropriate topology requires balancing the need for information exchange with the cost of communication.

These scalability trends are consistent with Amdahl's Law. As the number of islands increases, the quality improvement over Serial SA follows a diminishing return pattern: 28%, 37%, 42%, and 44% at 2, 4, 8, and 16 islands respectively. This suggests that the parallel fraction of AIPSA-GM, while substantial, is bounded by a serial component introduced by migration and communication overhead. In the fully connected topology, this effect is amplified by $O(n^2)$ communication cost, causing runtime to grow from 23.5s at 4 islands to 75.4s at 16 islands, confirming that communication overhead constitutes the dominant serial bottleneck at scale.

E. GPU Parallelism Tradeoffs

Experiment 6 shows that GPU-based parallelism provides consistent improvements in solution quality across both benchmarks, but the magnitude of improvement depends on problem scale and structure. On Rastrigin, the performance gain decreases as dimensionality increases, while on TSP, the gain increases with problem size.

In addition, the chain sweep results indicate that there is a tradeoff between the number of parallel chains and convergence depth. Increasing the number of chains improves diversity but reduces the number of iterations per chain, leading to diminishing returns beyond a certain point.

Finally, the throughput results demonstrate that GPU can execute significantly more iterations than CPU within the same time budget. However, the effective utilization of this computational advantage depends on how well the algorithm balances exploration and convergence.

F. Summary of Insights

Overall, the results suggest that the performance of parallel simulated annealing is governed by a combination of algorithmic design and system-level considerations. Migration strategy, temperature control, synchronization, topology, and hardware architecture all interact to determine the final outcome. No single design choice dominates across all settings, and effective optimization requires adapting these components to the characteristics of the problem.

IX. CONCLUSION & FUTURE WORK

In this work, we propose AIPSA-GM, a comprehensive framework for parallel simulated annealing that integrates guided migration, adaptive temperature control, asynchronous communication, and GPU acceleration. Our results confirm that these techniques significantly improve both solution quality and parallel efficiency.

All proposed hypotheses are validated through extensive experiments. Guided migration improves performance, adaptive temperature is effective but problem-dependent, asynchronous execution enhances scalability, topology influences

convergence, and GPU parallelism introduces meaningful but nuanced tradeoffs.

Despite these successes, several limitations remain. Python multiprocessing introduces overhead, and GPU performance is highly dependent on hardware characteristics. Future work includes exploring cluster-scale implementations, dynamic migration strategies, and heterogeneous island configurations.

X. REFERENCES

REFERENCES

- [1] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [2] N. Metropolis et al., "Equation of State Calculations by Fast Computing Machines," *Journal of Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, 1953.
- [3] M. Rucinski, D. Izzo, and F. Biscani, "On the impact of the migration topology on the Island Model," *Parallel Computing*, vol. 36, no. 10–11, pp. 555–571, 2010.
- [4] A. M. Ferreiro, J. A. García, J. G. López-Salas, and C. Vázquez, "An efficient implementation of parallel simulated annealing algorithm in GPUs," *Journal of Global Optimization*, vol. 57, pp. 863–890, 2013.

APPENDIX

Experiment 1

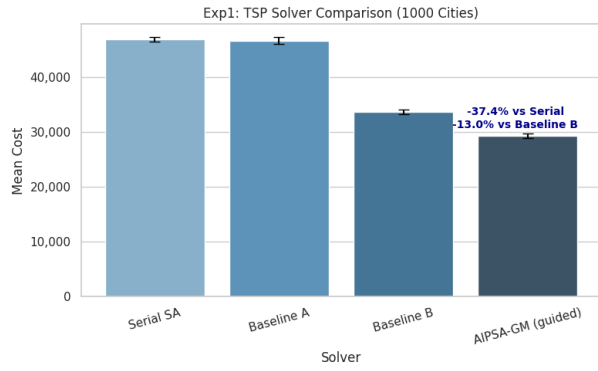


Fig. 1. TSP solver comparison (1000 cities).

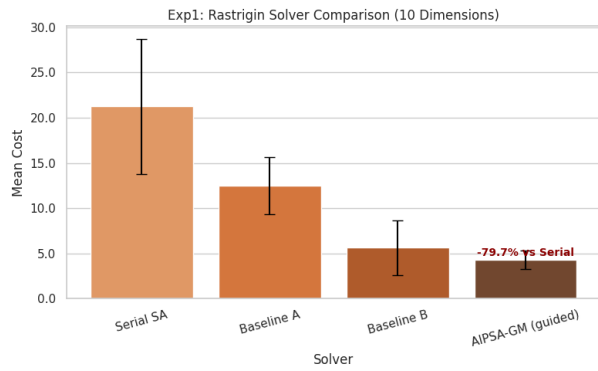


Fig. 2. Rastrigin solver comparison (10 dimensions).

Experiment 2

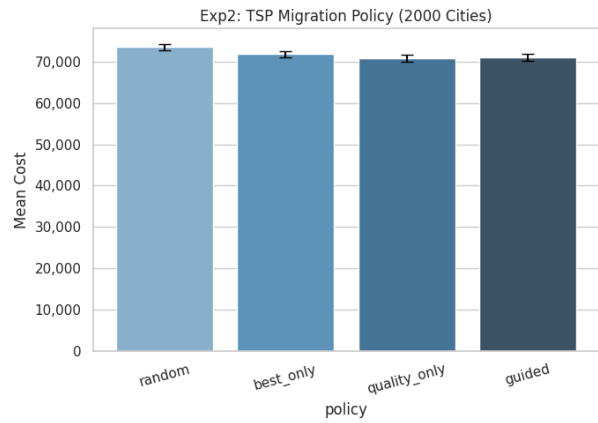


Fig. 3. Migration policy comparison on TSP (2000 cities).

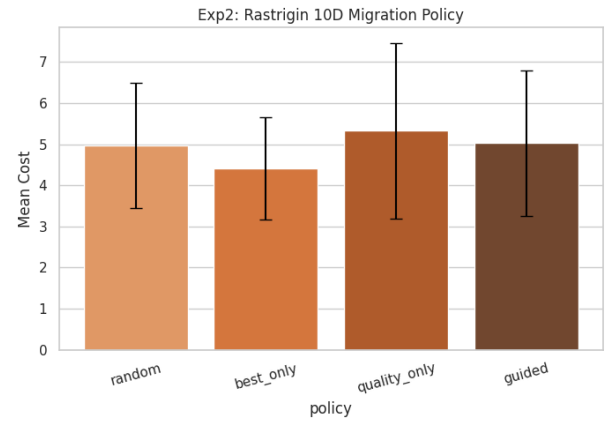


Fig. 4. Migration policy comparison on Rastrigin (10D).

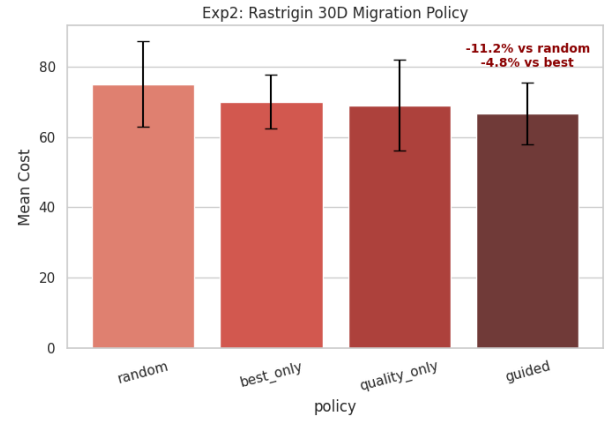


Fig. 5. Migration policy comparison on Rastrigin (30D).

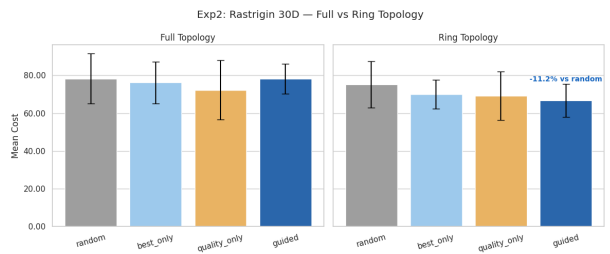


Fig. 6. Full vs ring topology on Rastrigin (30D).

Experiment 3

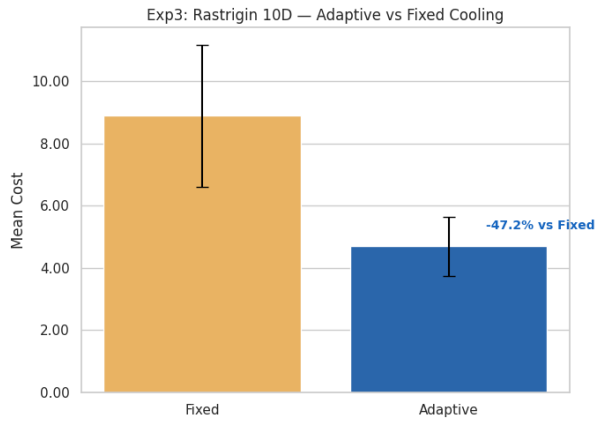


Fig. 7. Comparison between fixed and adaptive cooling on Rastrigin (10 dimensions). Error bars represent standard deviation across runs.

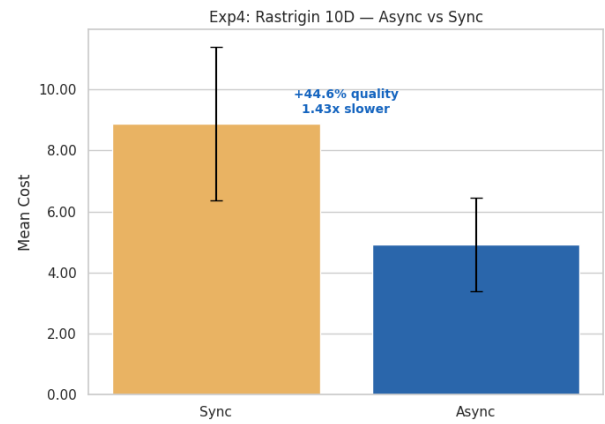


Fig. 10. Comparison between synchronous and asynchronous migration on Rastrigin (10 dimensions). Error bars represent standard deviation across runs.

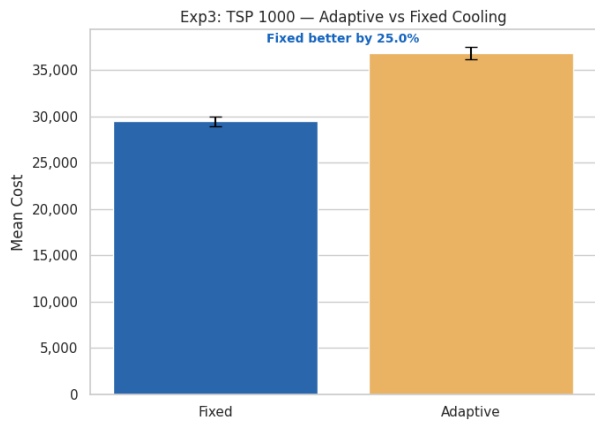


Fig. 8. Comparison between fixed and adaptive cooling on TSP (1000 cities). Error bars represent standard deviation across runs.

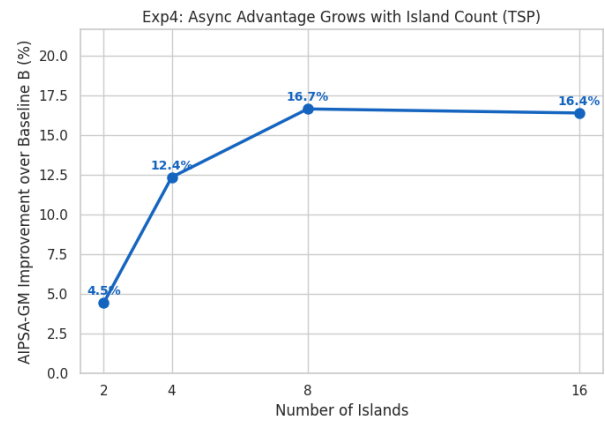


Fig. 11. Improvement of asynchronous migration over synchronous baseline as a function of the number of islands on TSP.

Experiment 4

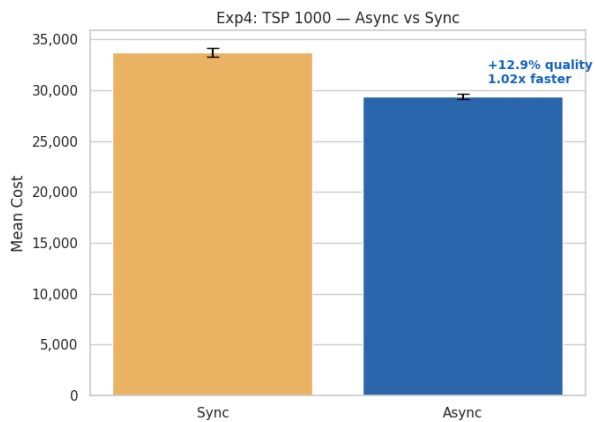


Fig. 9. Comparison between synchronous and asynchronous migration on TSP (1000 cities). Error bars represent standard deviation across runs.

Experiment 5

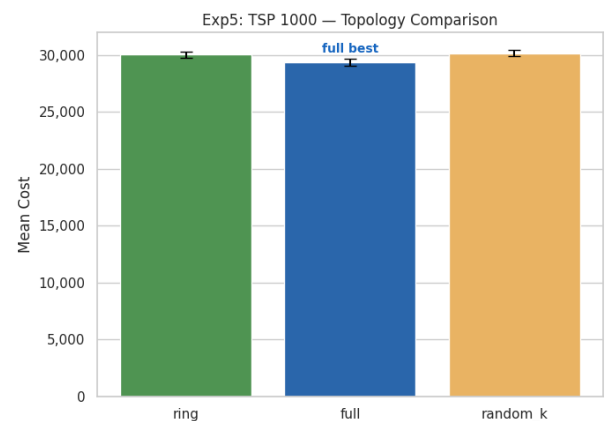


Fig. 12. Topology comparison on TSP (1000 cities)

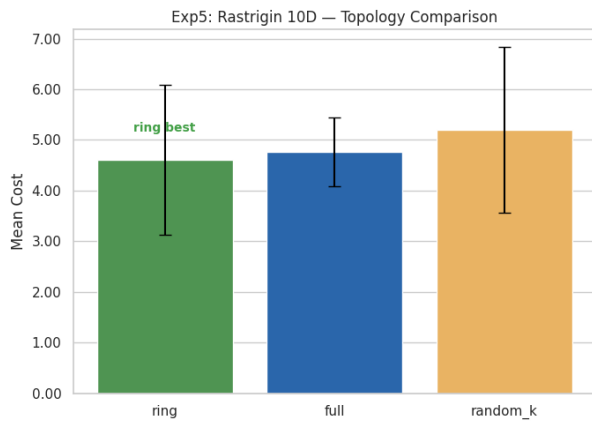


Fig. 13. Topology comparison on Rastrigin (10 dimensions).

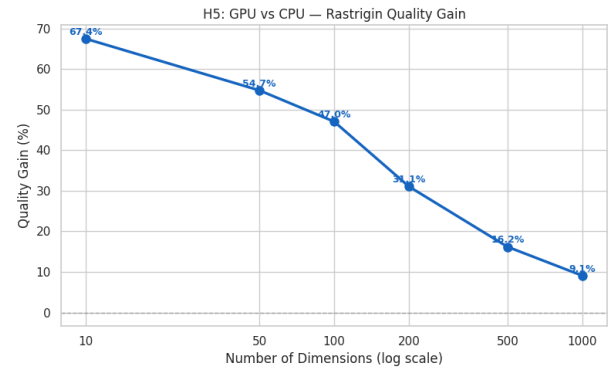


Fig. 16. Quality gain of GPU over CPU on Rastrigin across different dimensionalities.

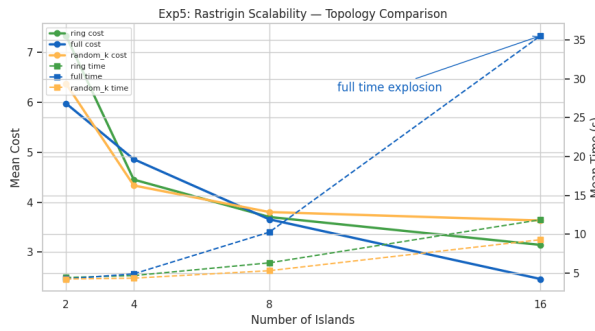


Fig. 14. Scalability of different topologies on Rastrigin as the number of islands increases. Solid lines indicate solution quality, and dashed lines indicate runtime.

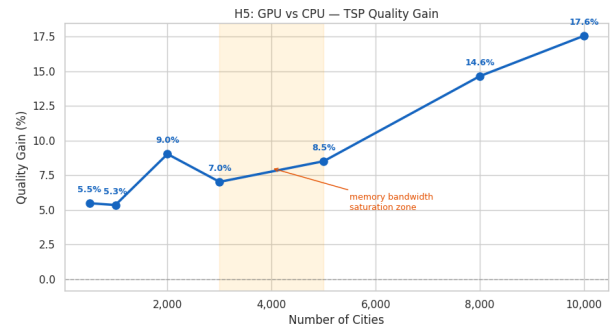


Fig. 17. Quality gain of GPU over CPU on TSP across different problem sizes.

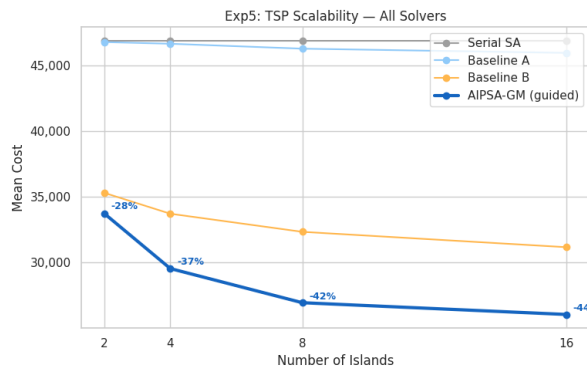


Fig. 15. Scalability of different solvers on TSP as the number of islands increases.

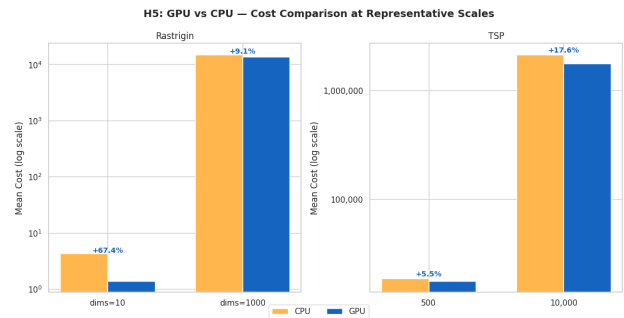


Fig. 18. Cost comparison between GPU and CPU at representative scales for Rastrigin and TSP.

Experiment 6

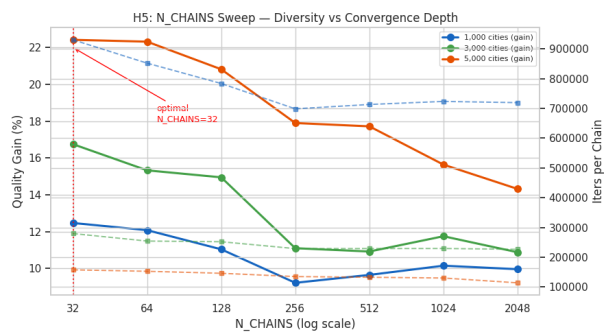


Fig. 19. Effect of the number of parallel chains on performance, showing the tradeoff between diversity and convergence depth.

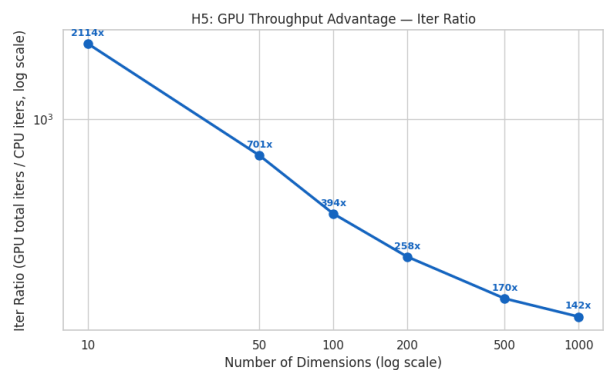


Fig. 20. Throughput advantage of GPU over CPU measured by iteration ratio across different Rastrigin dimensions.